The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 8 – Databases in Python

In the previous chapter, you learned how to store and retrieve data using files such as CSV, XML, and JSON. File-based storage is an important foundation and remains useful in many real-world scenarios. However, as applications grow, file storage begins to show limitations—especially when data becomes large, interconnected, or frequently modified. Searching through files can be slow, enforcing data rules is difficult, and maintaining relationships between different types of data quickly becomes complex.

Databases are designed to solve these problems. A database provides a structured environment for storing data, enforcing rules, and efficiently retrieving specific information. Relational databases, in particular, organize data into tables with clearly defined relationships between them. This structure allows applications to manage large amounts of data safely and consistently, while supporting features such as unique identifiers, constraints, and relationships between related records. For example, an online store can store product information once and connect it to many customer reviews without duplicating data.

In this chapter, you will be introduced to relational databases and the SQL language, and you will learn how to use a database directly from Python. Using SQLite and Python's built-in sqlite3 library, you will create tables, insert data, query records, update existing values, and safely delete information. By the end of this chapter, you will understand how databases fit into Python applications and how they extend the persistence concepts introduced earlier—laying the groundwork for more advanced data-driven and web-based systems in future courses.

8.1 Relational Databases and Why They Matter

A relational database stores data in a structured way and provides powerful tools for retrieving exactly the data you need. Think of it as a separate application that does all the searching, filtering and modification for you. It also provides a place to enforce rules—such as requiring unique IDs, preventing missing required values, and ensuring related records remain consistent.

Databases are handled in 'transactions' and when errors occur you can 'rollback' changes. This is key features to working with any database.

8.1.1 Tables, Rows, and Columns

Relational databases store data in tables, similar to spreadsheets, but with stronger rules and relationships.

- A column defines a field (such as name, price, or rating).
- A row represents a single record (such as one product or one review).
- A table represents a category of data (such as all products or all reviews).

When designing a system, a useful habit is to ask:
"What are my entities and what attributes do they have?"

For an online marketplace scenario:

- A Product entity might include: name, brand, price, and stock quantity.
- A Review entity might include: rating, title, comment, reviewer name

8.1.2 Primary Keys

A primary key uniquely identifies a row in a table. Primary keys are essential because they allow the database to distinguish one record from another.

Primary keys must be:

- Unique
- Not null
- Stable (they should not change over time)

In many systems, primary keys are artificial, meaning the key is created solely for identification. A common example is an auto-incrementing integer column named id. Artificial keys are popular because real-world fields like product names are not guaranteed to be unique and can change over time.

8.1.3 Foreign Keys and Relationships

Relational databases are powerful because tables can be related to one another.

A foreign key is a column in one table that refers to the primary key in another table. This is how databases represent relationships between different entities.

For example:

- One Product → Many Reviews

This relationship allows you to store product information once and store reviews separately, while still being able to connect them accurately.

8.1.4 Common Relational Databases

There are many relational databases in use today:

- SQLite (file-based, embedded, no server)
- MySQL / MariaDB (popular server-based databases)
- PostgreSQL (advanced open-source database)
- Oracle (enterprise/commercial database)
- SQL Server (Microsoft enterprise database)

Most relational databases use SQL and follow industry standards, but each has small differences in datatypes, features, and syntax. The good news is that once you learn the fundamentals, your knowledge transfers well between systems.

An important distinction is that SQLite is not a server. It stores the entire database as a single file (such as store.db), and your Python program connects directly to that file.

8.1.5 SQLite with this course

For this course, we will use SQLite because:

- No server setup is required
- It is simple and reliable
- It uses real SQL concepts
- It is easy to include in any project, no need to recreate the server on my end.

In order to work with SQLite you can use built in tools in PyCharm (which we will) but to get started and creating them separately we can use the DB Browser application.

You can find it here: https://sqlitebrowser.org/

It is a free high quality, visual, open source tool designed for people who want to create, search and edit SQLite database files. It works on Windows, Mac and most version of Linux. Feel free to download and install it on your computer.

8.2 Introduction to SQL and CRUD

SQL (Structured Query Language) is the standard language used to communicate with relational databases. Unlike Python, SQL is declarative: you describe what you want (for example, "give me all reviews for product 3"), and the database engine determines how to retrieve it efficiently.

Even though SQL has many advanced features, most applications rely heavily on a core set of operations that appear in almost every system. These operations are commonly summarized as CRUD:

- **C**reate → INSERT (add new rows)
- **R**ead → SELECT (retrieve rows)
- **U**pdate → UPDATE (modify existing rows)
- **D**elete → DELETE (remove rows)

You will see CRUD concepts everywhere: online stores, student systems, banking apps, healthcare software, and social media platforms all revolve around creating, reading, updating, and deleting data.

In this chapter, we'll use a simplified "online store" scenario: Products and Reviews (similar to Amazon-style product pages).

8.2.1 Creating Tables and Choosing Datatypes

Before you can store data, you need tables. Tables are created with `CREATE TABLE`. There are other properties we can do like often times we add `IF NOT EXISTS` so that we don't get errors for trying to create the table again.

In SQL, a table definition includes the column names, the datatype for each column, and constraints such as `PRIMARY KEY`, `NOT NULL`, and `FOREIGN KEY`. When defining columns each one should be on its own line and ends with a comma as long as you keep defining columns. The last column you don't need a comma as it is the end. All column names are in parentheses and ends with a semi colon.

SQLite supports common datatypes such as:

- INTEGER – whole numbers
- REAL – decimals
- TEXT – strings
- BLOB – raw binary data

Different databases implement datatypes differently. SQLite is more flexible than many server databases, but datatypes should still be chosen carefully because they communicate with intent and help keep your data consistent.

Note on Date Comparison

One thing you will find in the class examples and needed in the assignments is the need to have date comparison. Many other databases have a date or date and time comparison. Again as shown above, SQLite doesn't. For this though we can use the TEXT datatype as long as we use the correct format for the date.

"YYYY-MM-DD HH:MM:SS.MMMMM"

This is that we have a four digit year, 2 digit month, 2 digit day and the time as well follows. If just dealing with the date, extract the time, and vice versa for just dealing with time. One thing that computers use for comparing text is lexicographical ordering. This is basically the dictionary style ordering. It goes from left to right checking the order of text.

Numbers are ordered least to greatest of course so having a date in YYYY-MM-DD format can be used checking if a value is in a range for greater than, less than, etc. We always check the year first, then we check the month, then lastly the day. So that should make sense to have in that order. Most programming languages (Python including) will save out the date in that format by default. So we will be all set with saving our dates in that format.

Product table:

```
CREATE TABLE IF NOT EXISTS product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    brand TEXT NOT NULL,
    price REAL NOT NULL,
    stock_qty INTEGER NOT NULL
);
```

Review table (with foreign key):

```
CREATE TABLE IF NOT EXISTS review (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_id INTEGER NOT NULL,
    rating INTEGER NOT NULL,
    title TEXT NOT NULL,
    comment TEXT NOT NULL,
    reviewer TEXT NOT NULL,
    FOREIGN KEY (product_id) REFERENCES product(id)
);
```

8.2.2 Reading Data with SELECT

SELECT is used to retrieve data. In practice, you will use SELECT more than any other SQL command because reading data is the foundation of reports, search screens, dashboards, and product page views. The basic layout is as follows:

SELECT *col_1, col_2, .. col_n* FROM *table_name*;

For selecting all the columns in a table, we use the asterisk (*) as a 'wildcard' meaning all. Selecting all columns:

```
SELECT * FROM product;
```

	id	name	brand	price	stock_qty
1	1	Bluetooth Headphones	SoundMax	59.99	25
2	2	USB-C Charging Calbe	TechLine	12.49	100
3	3	Wireless Mouse	ClickPro	24.99	40

```
Execution finished without errors.
Result: 3 rows returned in 4ms
At line 1:
SELECT * FROM product;
```

Selecting specific columns:

```
SELECT name, price FROM product;
```

	name	price
1	Bluetooth Headphones	59.99
2	USB-C Charging Calbe	12.49
3	Wireless Mouse	24.99

```
Execution finished without errors.
Result: 3 rows returned in 3ms
At line 1:
SELECT name, price FROM product;
```

Filtering rows with `WHERE`:

```
SELECT * FROM product
WHERE stock_qty > 30;
```

	id	name	brand	price	stock_qty
1	2	USB-C Charging Calbe	TechLine	12.49	100
2	3	Wireless Mouse	ClickPro	24.99	40

```
Execution finished without errors.
Result: 2 rows returned in 5ms
At line 1:
SELECT * FROM product
WHERE stock_qty > 30;
```

Sorting results use either `ASC` or `DESC` with `ORDER BY` stating what column:

```
SELECT id, name, price FROM product
ORDER BY price DESC;
```

	id	name	price
1	1	Bluetooth Headphones	59.99
2	3	Wireless Mouse	24.99
3	2	USB-C Charging Calbe	12.49

```
Execution finished without errors.
Result: 3 rows returned in 10ms
At line 1:
SELECT id, name, price FROM product
ORDER BY price DESC;
```

Selecting reviews for a specific product:

```
SELECT rating, title, comment, reviewer
FROM review
WHERE product_id = 1;
```

	rating	title	comment	reviewer
1	5	Amazing sound	Battery lasts all week.	Ava
2	4	Good value	Comfortable but slightly tight.	Noah

```
Execution finished without errors.
Result: 2 rows returned in 7ms
At line 1:
SELECT rating, title, comment, reviewer
FROM review
WHERE product_id = 1;
```

Checking a range value using AND

```
SELECT rating, title, comment, reviewer
FROM review
WHERE rating >= 2 AND rating <= 4;
```

	rating	title	comment	reviewer
1	4	Good value	Comfortable but slightly tight.	Noah
2	3	Decent mouse	Clicks are loud but repsonsive.	Lucas

```
Execution finished without errors.
Result: 2 rows returned in 6ms
At line 1:
SELECT rating, title, comment, reviewer
FROM review
WHERE rating >= 2 AND rating <= 4;
```

Or use BETWEEN

```
SELECT rating, title, comment, reviewer
FROM review
WHERE rating BETWEEN 2 AND 4;
```

	rating	title	comment	reviewer
1	4	Good value	Comfortable but slightly tight.	Noah
2	3	Decent mouse	Clicks are loud but repsonsive.	Lucas

```
Execution finished without errors.
Result: 2 rows returned in 7ms
At line 1:
SELECT rating, title, comment, reviewer
FROM review
WHERE rating BETWEEN 2 AND 4;
```

8.2.3 Aggregate Functions: COUNT, AVG, SUM

Databases can calculate summaries directly, which is often faster and cleaner than loading all rows into Python first.

COUNT():

```
SELECT COUNT(*) FROM review
WHERE product_id = 1;
```

	COUNT(*)
1	2

```
Execution finished without errors.
Result: 1 rows returned in 4ms
At line 1:
SELECT COUNT(*) FROM review
WHERE product_id = 1;
```

AVG():

```
SELECT AVG(rating) FROM review
WHERE product_id = 1;
```

	AVG(rating)
1	4.5

```
Execution finished without errors.
Result: 1 rows returned in 12ms
At line 1:
SELECT AVG(rating) FROM review
WHERE product_id = 1;
```

SUM():

```
SELECT SUM(stock_qty) FROM product;
```

	SUM(stock_qty)
1	165

```
Execution finished without errors.
Result: 1 rows returned in 5ms
At line 1:
SELECT SUM(stock_qty) FROM product;
```

8.2.4 Inserting Data with INSERT

`INSERT` adds new row in the table. You may need to list out the columns for inserting or if you know you are inserting into all the columns you can omit these. In our example since we have an 'id' column that auto increments we need to spell out what columns we are inserting data into.

Inserting a product:

```
INSERT INTO product (name, brand, price, stock_qty)
VALUES ('Bluetooth Headphones', 'SoundMax', 59.99, 25);
```

Inserting a review:

```
INSERT INTO review (product_id, rating, title, comment, reviewer)
VALUES (1, 5, 'Great sound!', 'Battery lasted all week.', 'Ava');
```

8.2.5 Updating Data with UPDATE

`UPDATE` modifies existing rows. We must define the table to update, use the SET command to define the columns (multiple separated by commas) and their values to update to with '='. Use the WHERE clause to define a condition. You can modify every row by not have WHERE or multiple rows if you are doing something like relational operation(say all the prices greater than $25)

Update the stock_qty of a product via their id

```
UPDATE product
SET stock_qty = 20
WHERE id = 1;
```

Update all the prices by increasing 5%

```
UPDATE product
SET price = price * 1.05;
```

8.2.6 Deleting Data with DELETE

`DELETE` removes rows from a table. Always be careful to include a `WHERE` clause. If you forget the `WHERE` clause, all the data in the table will be deleted. Often in a production environment you won't have access to delete.

```
DELETE FROM review
WHERE id = 10;
```

When deleting data, we need to be conscious of the relationships in our database. If you try to delete an item that has a primary key used as foreign keys in a separate table you will get a constraint error.

To solve this, you can set up `ON DELETE CASCADE` attribute on the foreign key reference in the table creation. The other option is to first delete the rows in the "child" table with the foreign key reference then delete in the main table.

For example, in our products and reviews. Delete all rows in the reviews with a matching product_id. Then delete via that same id in the products table.

Deleting a table entirely:

```
DROP TABLE review;
```

One last note on deleting. In a business environment usually, there are "users" defined with set privileges including if you can delete. This is often locked down by the database administrator.

Another option to think about is not deleting but marking rows as "active" or "inactive" so that you keep the reference to old data. Another word used might be "archived."

How would we "mark" them? Well, we can take rows out of one table and move them to a different table that is our inactive/archived table. Or we could add a column to our table to signify active (maybe a boolean value of True/False (1 or 0 in SQLite). Whichever way you choose depends on system design and business rules.

We will explore some of this in the later sections when we implement deleting in our Python code.

8.2.7 SQL Coding Style

While not required most SQL developers follow a style that makes it easier to read.

- Typically on SQL commands we put in all caps.
- Usually table names and columns are in lower case.
- If a column contains two words we put in an underscore to separate. (column_name).
- Usually we separate out commands for SQL on new lines.
    - ```
      SELECT *
      FROM table_name
      WHERE id = 7
      ORDER BY price ASC;
      ```
    - Sometimes FROM can be on the same line as SELECT especially using '*' to select all.
- Each SQL command ends with a semi colon.
    - Only when direct coding with SQL, when using a programming language, we don't need them.
      ```

8.3 Using SQLite in Python with sqlite3

Python includes a built-in library called `sqlite3`, which allows your programs to connect to SQLite database files and execute SQL statements directly. This makes SQLite a natural choice for learning database concepts without introducing additional dependencies.

8.3.1 Connecting to a Database

Connecting to a SQLite database is done using the `connect()` function. If the database file does not exist, SQLite automatically creates it.

```
import sqlite3
conn = sqlite3.connect("store.db")
```

The connection object represents an active link to the database. All database operations flow through this connection.

8.3.2 Helpful connect() Options

In most introductory programs, you only need to supply the database filename. However, `connect()` supports optional parameters that are commonly seen in real applications.

- `timeout` – how long SQLite waits if the database is locked
- `detect_types` – helps SQLite parse specialized types like dates
- `check_same_thread` – used in threaded applications
- `isolation_level` – controls transaction behavior

```
conn = sqlite3.connect("store.db", timeout=10)
```

8.3.3 Cursor Objects

A `cursor` is the object used to execute SQL statements and retrieve results from the database. Think of the cursor as the command interface between Python and SQL.

```
cursor = conn.cursor()
```

8.4 Creating and Initializing a SQLite Database

Before a program can store or retrieve data from a database, the database itself must exist and contain the appropriate tables. In SQLite, creating a database is a lightweight operation: connecting to a database file automatically creates it if it does not already exist.

This section walks through the full lifecycle of database setup, from creating the database file to inserting initial data and safely closing the connection. Understanding this process is essential because it mirrors how real applications initialize their persistence layer.

8.4.1 Creating a Database File with connect()

SQLite databases are stored as files on disk. When you call sqlite3.connect(), one of two things happens:

• If the file exists, SQLite opens it.

• If the file does not exist, SQLite creates it automatically.

```
import sqlite3

conn = sqlite3.connect("store.db")
```

At this point, the database file exists on disk, but no tables or data have been created yet.

This behavior makes SQLite ideal for learning environments, small desktop applications, prototype systems, and course projects because no separate database server is required.

8.4.2 Getting a Cursor Object

Once connected, you must create a cursor. The cursor is responsible for executing SQL statements and retrieving results from the database.

```
cursor = conn.cursor()
```

You can think of the cursor as the SQL command channel between Python and the database. All SQL execution goes through a cursor object.

8.4.3 Enabling Foreign Key Support in SQLite

Unlike many server-based databases, SQLite does not enable foreign key constraints by default. If your database design relies on foreign keys, you must explicitly enable them for each connection.

```
cursor.execute("PRAGMA foreign_keys = ON;")
```

This ensures that reviews cannot reference non-existent products and that referential integrity rules are enforced in our example.

8.4.4 Creating Tables Safely with IF NOT EXISTS

Table creation is done using CREATE TABLE. Running the same SQL multiple times would normally cause errors, so SQL provides the IF NOT EXISTS clause to make table creation safe during application startup.

```
CREATE_PRODUCT_TABLE = """
CREATE TABLE IF NOT EXISTS product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    brand TEXT NOT NULL,
    price REAL NOT NULL,
    stock_qty INTEGER NOT NULL
);
"""

CREATE_REVIEW_TABLE = """
CREATE TABLE IF NOT EXISTS review (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_id INTEGER NOT NULL,
    rating INTEGER NOT NULL,
    title TEXT NOT NULL,
    comment TEXT NOT NULL,
    reviewer TEXT NOT NULL,
    FOREIGN KEY(product_id) REFERENCES product(id)
);
"""
```

8.4.5 Executing Table Creation SQL

The execute() method runs one SQL command at a time. This keeps execution predictable and avoids partial failures.

```
cursor.execute(CREATE_PRODUCT_TABLE)
cursor.execute(CREATE_REVIEW_TABLE)
```

8.4.6 Inserting Initial Product Data

After creating tables, it is common to insert seed data so the database starts in a usable state.

```
INSERT_PRODUCTS = """
INSERT INTO product (name, brand, price, stock_qty)
VALUES (?, ?, ?, ?)
"""

products = [
    ("Bluetooth Headphones", "SoundMax", 59.99, 25),
    ("USB-C Charging Cable", "TechLine", 12.49, 100),
    ("Wireless Mouse", "ClickPro", 24.99, 40)
]

for p in products:
    cursor.execute(INSERT_PRODUCTS, p)
```

8.4.7 Inserting Related Review Data

```
cursor.execute("SELECT id FROM product ORDER BY id")
product_ids = [row[0] for row in cursor.fetchall()]

INSERT_REVIEWS = """
INSERT INTO review (product_id, rating, title, comment, reviewer)
VALUES (?, ?, ?, ?, ?)
"""

reviews = [
    (product_ids[0], 5, "Amazing sound", "Battery lasts all week.", "Ava"),
    (product_ids[0], 4, "Good value", "Comfortable but slightly tight.", "Noah"),
    (product_ids[1], 5, "Perfect cable", "Works exactly as expected.", "Mia"),
    (product_ids[2], 3, "Decent mouse", "Clicks are loud but responsive.", "Lucas")
]

for r in reviews:
    cursor.execute(INSERT_REVIEWS, r)
```

8.4.8 Committing Changes to the Database

```
conn.commit()
```

Calling `commit()` permanently saves changes to the database. If the program crashes before committing it, SQLite can roll back the transaction automatically.

8.4.9 Closing the Connection

```
conn.close()
```

Closing the connection releases file locks, ensures data integrity, and prevents resource leaks.

8.5 Proper SQL Use in Coding

We have a couple issues before getting into executing specific SQL commands in our code. That is making sure to use parameterized statements and handling exceptions that can occur when performing inserts, updates, and deletes.

8.5.1 Parameterized Statements and SQL Injection

SQL injection is a vulnerability that happens when a program builds SQL using raw user input. If a user can inject SQL into an input field, they may be able to:

- bypass login checks
- delete tables
- extract confidential data

Example of the dangerous approach:

```
name = input("Product name: ")
sql = f"SELECT * FROM product WHERE name = '{name}'"
cursor.execute(sql)
```

If a user enters something malicious like:

```
' OR 1=1 --
```

the SQL becomes:

```
SELECT * FROM product WHERE name = '' OR 1=1 --'
```

`OR 1=1` is always true, so the query returns all products. In more serious cases, attackers can attempt to execute destructive commands.

The safe solution: parameterized queries

Parameterized queries keep the SQL command separate from the data:

```
cursor.execute("SELECT * FROM product WHERE name = ?",(name,))
```

SQLite treats the parameter as data, not executable SQL. Note that data needs to be a tuple. So if it is even a single element within you need a comma. Note `(name,)` adding the comma makes it a tuple.

You should use parameterized queries for:

- SELECT with user-provided filters
- INSERT
- UPDATE
- DELETE

8.5.2 Exception Handling with SQL

Database operations are one of the most failure-prone parts of an application. Even when your SQL is correct having invalid user input, invalid foreign keys or other constraint violations can cause errors.

To mitigate these, we should implement try statements in our code.

- Gracefully handle fails instead of crashing
- Roll back changes if something goes wrong
- Return useful feedback to the rest of the program

We really focus on the INSERT, UPDATE and DELETE statements when implementing a try/except. Also, something to keep in mind is that when you use a try/except the error doesn't get printed out so you should at least debug catch the exception and print it out. That way you can see if there are any basic SQL errors in our statements. After that and in the insert, modify and delete sections below you will see we don't print the exceptions as this code doesn't make errors after we have debugged.

The basic operation:

1. Get the connection and cursor (from a connection factory talked about next)
2. use the 'try' to execute SQL, commit the changes and return a value that represents "success"
3. Use except to catch the error, call the rollback() on the connection to undo changes, and return a values that means "unsuccessful." Here you will also see we are printing out the error for debugging purposes.
4. Use the finally block to call close() for closing the connection no matter if we were successful or had an error.

```
def some_function(object : Object):
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("Some SQL command here", (parameterized values,))
        conn.commit()
        return some_value_success
    except Exception ex:
        conn.rollback()
        print(ex)
        return return_value_unsuccessful
    finally:
        conn.close() # close no matter error or not
```

Remember that in a system we typically have our "presentation layer" that does all our output/printing. These functions return a value that we handle in if statements to decide what we want to print.

Once we get the code working with no SQL errors we can comment out the print(). This would be ok to keep if you want to log information, especially if you have a GUI based application, you could still show errors here. The normal user won't see the console output unless it is a console based UI.

In the functions that follow under the insert, modify and delete you will see that there is no variable for the exception and printing of it. Just make note of that and if your program isn't working correctly as you create it, that you should be printing the exceptions to see the issues.

8.6 Connection Factory

As soon as your program starts interacting with a database in multiple places you will notice a repeated pattern:

1. Connect to the database
2. Create a cursor
3. Turn on foreign key enforcement(for SQLite)
4. Run SQL
5. Commit if needed
6. Close the connection

If you copy and paste that setup code into every functionality you code to perform CRUD operations it makes it harder and harder to maintain. Repetition of code increases the chance that something will be inconsistent.

- One function might forget to enable foreign keys.
- Another might use a different timeout.
- A third might forget to close the connection.

When the "setup" logic is repeated everywhere, small bugs become difficult to track down because there isn't one clear place to fix the issue.

A better approach is to create one function responsible for returning a properly configured connection to a database. Then every database function uses that helper. This gives your code

- **Consistency:** every connection uses the same settings.
- **Maintainability:** if you change the database filename or timeout later, you change it in one place.
- **Readability:** your CRUD functions stay focused on their real purpose (fetch, insert, update, delete) instead of repeating boilerplate setup.

This enters the factory pattern in software design. A factory function or object has the job to create and return other objects in a consistent way.

Since we are making the same connection repeatedly we can implement a helper function as a factory to get a connection to our database every time.

```
import sqlite3

def get_connection(db_name: str = "store.db") -> sqlite3.Connection:
    conn = sqlite3.connect(db_name, timeout=10)
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")
    return conn
```

Here we are accepting the name of the database to connect to, or defaulting to a value then making the connection, setting the foreign key specifics to SQLite and returning that connection.

Another option could be having a constant value on the module that is the database name, getting it from system arguments sent to it upon running, or even an environment variable set on the system the application is running on.

```
import sqlite3

DB_URL = "store.db"

def get_connection()-> sqlite3.Connection:
    conn = sqlite3.connect(DB_URL,)
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")
    return conn
```

8.7 Querying Data with fetchall, fetchone, fetchmany

To query our database we need to `execute()` a `SELECT` command that returns our data on the `cursor` object. We can then use `fetchall()`, `fetchone()` or `fetchmany()`.

Data is typically returned as text data and needs to be converted to an object that is useful to our program. In the following sections we will use the following objects for Products and Reviews.

```python
class Review:
    def __init__(self, review_id: int=0, product_id: int, rating: int,
                 title: str, comment: str, reviewer: str):
        self.review_id = review_id
        self.product_id = product_id
        self.rating = rating
        self.title = title
        self.comment = comment
        self.reviewer = reviewer

class Product:
    def __init__(self, product_id: int=0, name: str, brand: str,
                 price: float, stock_qty: int, reviews=None):
        self.product_id = product_id
        self.name = name
        self.brand = brand
        self.price = price
        self.stock_qty = stock_qty
        self.reviews = reviews if reviews is not None else []
```

8.7.1 Getting all rows

The most basic of all the functions is the `fetchall()`. Once you execute a `SELECT` query you can all the `fetchall()` on the cursor object. That then returns a list of tuple values. Remember that tuples are immutable(meaning they can't be changed) which makes perfect sense for our data as it is read only data coming from a database.

You will then take that data loop through it and convert the rows into a list of objects (or whatever conversion you want for your system).

```python
def get_all_products() -> list[Product]:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM product")
    rows = cursor.fetchall()

    products = []
    for row in rows:
        reviews = get_product_reviews(row[0])
        products.append(Product(row[0], row[1], row[2], float(row[3]),
                                int(row[4]), reviews))

    conn.close()
    return products
```

Here we are directly accessing index locations of the tuple. You will need to make sure you understand the order of your data coming from the table in your database in order to perform this action. Alternatively, your select could be the column names in the order you want along with setting the values of a tuple to variables.

You also see this `get_product_reviews()` function. Well on our products we need to get all the reviews, convert to a `Review` object and attach it to the `Product`.

```
def get_product_reviews(product_id: int ) -> list[Review]:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
      "SELECT * FROM review WHERE product_id = ? ORDER BY id", (product_id,))

    rows = cursor.fetchall()

    reviews = [Review(r[0], r[1], int(r[2]), r[3], r[4], r[5]) for r in rows]

    conn.close()
    return reviews
```

If you are fluent in SQL you may think "Why didn't he just do a JOIN?" You are correct, the best way potentially could be to use a JOIN statement here. I decided to just focus on basic SQL here and programming logic as this isn't a full SQL/database course.

8.7.2 Getting one row

If you are searching for one entry in the database or you just want to get the first element in the data that was returned we can use the `fetchone()` record in the database table.

You will then need to translate it to an object or however you are storing/using the data in your system.

```
def get_product_by_id(product_id: int) -> Product | None:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM product WHERE id = ?",(product_id,))
    row = cursor.fetchone()
    conn.close()

    if row is None:
        return None

    return Product(row[0], row[1], row[2], float(row[3]),
                           int(row[4]), get_product_reviews(row[0]))
```

8.7.3 Limited size of data

In many real-world systems, tables may contain millions of rows. Pulling all rows at once wastes time and memory. We can solve this by showing a limited number of results. For this the `fetchmany(size)` function takes on the number of rows of data to get.

If you have ever heard of "pagination" this is a similar operation. Where websites show pages of items and at the bottom or somewhere there is navigation to go to the next "page." Think like a google search or when you are on a shopping site and as you scroll to the bottom more items automatically load.

One big note, make sure your SQL command orders the data as the database could return the same data in a mixed order from the last query you did.

```
def get_products_size(page_size: int) -> list[Product]:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM product ORDER BY id")
    rows = cursor.fetchmany(page_size)

    products = [Product(r[0], r[1], r[2], float(r[3]),
                        int(r[4]), get_product_reviews(r[0])) for r in rows]
    conn.close()
    return products
```

You will see here we are throwing in some `Pythonic` operations by using a comprehension to build our list of products.

8.8 Inserting Records

To insert we must create the SQL statement according to how your table is designed. In this case the product_id is auto generated from the database, so we don't need to insert it and must define all the columns to insert values into.

Often we pass an object to the function as we do below. But if it is a small enough table or you don't have a product in your system then you can pass parameters directly (here would be name, brand, price, stock_qty as separate values).

```python
def insert_product(product : Product -> int:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
          "INSERT INTO product (name, brand, price, stock_qty) VALUES (?, ?, ?, ?)",
          (product.name, product.brand, product.price, product.stock_qty))

        conn.commit()
        return int(cursor.lastrowid)
    except Exception:
        conn.rollback()
        return -1
    finally:
        conn.close() # close no matter error or not
```

Note that this function gets the `lastrowid` which is the `product_id` since the database handles the id auto incrementing. You could also make this function return `True/False` or the product that was entered in successfully(making sure to assign `product_id`). Lastly you will see we rollback any changes to get SQLite out of a failed state. Here if an error occurs we are return -1 to signify an error coming back.

We also need to insert reviews. This is the same concept of getting a `Review` object in the parameters, then connect and using a try/except attempt to make the changes.

```python
def insert_review(review : Review) -> int:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
        """ INSERT INTO review (product_id, rating, title, comment, reviewer)
            VALUES (?, ?, ?, ?, ?)""",
            (review.product_id, review.rating, review.title,
                                  review.comment, review.reviewer))
        conn.commit()
        return int(cursor.lastrowid)
    except Exception:
        conn.rollback()
        return -1
    finally:
        conn.close() # close no matter error or not
```

Since we have foreign key for `product_id`, the database enforces that this must match an id of a product in the products table. If it does not an error occurs and we rollback our changes. Otherwise, it will be inserted correctly and return the id of the review.

8.9 Modifying Records

When thinking about your system design you could have a rather small system where you are updating one or just a couple of columns. You could also have a larger system that allows for updating every column of a row in the database table. Some design options follow depending on you system.

8.9.1 Simple Updates of One Item

When you just need to update one or a couple columns you can create a simple function(s) to perform those updates. One thing in our example system for this chapter on a store is that when we get new inventory in, we need to update our on hand stock. Here is a function that will update that stock to the sent value.

Note that this function is return either `True` or successful update or `False` if not. If we successfully make an update we are grabbing the `rowcount` value. This is on the `cursor` and is the number of rows that we successfully updated. We should have only one row at the `product id` value and if we find the item and do the update then that row count should be 1.

Here we return a boolean expression making sure that it is 1, if it isn't then `False` gets returned. Same goes in the except if we have an error we return `False`.

```
def update_product_stock(product_id: int, new_stock_qty: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
            "UPDATE product SET stock_qty = ? WHERE id = ?",
            (new_stock_qty, product_id))
        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

The above function does a direct update with the sent value. We could also ADD to the on hand stock as another option.

```
def add_product_stock(product_id: int, add_qty: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
            "UPDATE product SET stock_qty = stock_qty + ? WHERE id = ?",
            (add_qty, product_id))
        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

8.9.2 Simple Update of Multiple Items

The previous examples showed the update of exactly one row as we are matching the primary key for the '`id`' of the row. We can also perform updates on multiple rows or all the rows in our tables.

For updating multiple rows your `WHERE` clause would be checking for a range of values. (Maybe you are checking all items with a `price > 100` to update some value.) For updating the entire table, you would omit the `WHERE` altogether.

For example, this loops through all the products in the table and `UPDATE` the price based on the percentage sent to it(.05 -> 5% increase, etc.). I put in some extra checks as well like making sure percentage is greater than 0 to make sure no errors/bad modifications happen. We can't control the decimal count to track in a column like other databases, so we use the `ROUND()` function to round to 2 decimal places.

For our boolean return we check here in the return for the `rowcount` value but make sure it is greater than equal to 1. Meaning we have updated at least one row in the table. If the table is empty, we can't update anything.

```
def raise_product_price_percentage(percentage : float) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    if percentage <= 0:
        return False # not update/error on percentage

    try:
        cursor.execute(
         "UPDATE product SET price = ROUND(price * ?, 2)", (1 + percentage,))

        conn.commit()

        return cursor.rowcount >= 1

    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

8.9.3 Large Updates Made Simpler

What if our system here allowed for us to directly update the name, brand, price, or on a review modify any of those values. If you were to think in simpler terms like the previous sections then you would look at creating a function for each way of updating.

This means you would have some of the following functions

```
def update_product_name(product_id : int, name : str) -> bool:
def update_product_brand(product_id : int, brand : str) -> bool:
def update_product_price(product_id : int, price : float) -> bool:
def update_review_rating(review_id : int, rating : int) -> bool:
def update_review_title(review_id : int, title : str) -> bool:
def update_review_description(review_id : int, description : str) -> bool:
```

This is a lot of functions with a lot of the same code. You connect to the database, get the cursor, then call a `UPDATE` command on the same table just updating one column.

This is fine, but it's a lot of extra code. A simpler way would be to have one update function (per the table/object you are updating), on your UI side do the modifications on the current object instance as you would normally, then send that instance to your update function. This function would then perform the `UPDATE` on the entire object for the primary key we are at.

"But what if we are only updating one column?" you may ask. If you think logically through it, we are updating the value, but the values we don't change will write over themselves and they stay the same. You may think it is redundant to overwrite the same data, but we would have a lot of redundant code and extra functions if we kept each operation separate.

You can still keep some of those simple update functions available. Especially like the updating of the stock_qty of our products. We may not want to give someone a way to update the stock_qty and other columns if their job is just inventory management. Same things goes with price updates. Again, all in the design of the system you are creating.

Here we have a function to update the entire Product object on our tables(at least the columns we want to allow for updating). Then doing the same for Reviews. Both functions are only excepting an object in the parameters, and we grab the properties to use in our update.

```
def update_product(product : Product) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
         """UPDATE product SET name = ?, brand = ?, price = ?, stock_qty = ?
         WHERE id = ? """, (product.name, product.brand, product.price,
             product.stock_qty, product.id))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Can't change the reviewer, so only updating the rating, title and comment.

```
def update_review(review : Review) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
         "UPDATE review SET rating = ?, title = ?, comment = ? WHERE id = ?",
            (review.rating, review.title, review.comment, review.id))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

8.10 Deleting Records

When deleting data there are two approaches that we will explore. One is by just doing a simple delete or a row (or multiple rows) at a time. The other option is to implement the use of keeping all data but making it "active" vs "inactive" or performing what you might call a "soft delete." Depending on regulations of the industry that you are building the system, you may have to keep the data you have for a certain period of time.

No matter what form we are doing there is some deleting involved. As discussed previously in the SQL section, we need to make sure that we are using the `WHERE` clause to limit what we are deleting.

8.10.1 Simple Deletes

For the example below we are deleting a review out of the table via the sent ID. On the UI side we would have some look up of the review, have the review as an instance of the Review object, then get the id sending over to this function.

```
def delete_review(review_id: int)-> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("DELETE FROM review WHERE id = ?", (review_id,))
        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

In the previous code we focused on deleting a review here. I didn't start with deleting a product as we have to revisit what was discussed previously with thoughtful deleting when foreign keys are involved.

If you set up your database to have the `ON DELETE CASCADE` when connecting your foreign keys, then the following isn't required and you can simply delete the product and it will delete the connected reviews.

But if your database is “simple” like I originally setup then you have to perform two deletes. One on the reviews table and then delete the product on that table.

Delete one product by ID and handling reviews

```
def delete_product(product_id: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("DELETE FROM review WHERE product_id = ?", (product_id,))
        reviews_rows = cursor.rowcount
        cursor.execute("DELETE FROM product WHERE id = ?", (product_id,))
        conn.commit()

        return (reviews_rows + cursor.rowcount) >= 1

    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Each time you `execute()` a command the `rowcount` property is to the most recent. Since we did two `executes()` we need to save the first, then I am adding that to the most recent `execute()` and checking if it is greater than or equal to one.

If there is a product that gets deleted with no reviews, the row count is one. If there are multiple reviews deleted along with a product, then that value is greater than one, so we are checking for both by using the greater than equal to in our boolean expression that is being returned.

8.10.2 Soft Delete – Moving Row to Table - Archive

One of the ways we can perform what I like to call a soft delete is by taking a row in the table and moving it to another table that is our inactive or archived data. The basic logic for this is we need to insert into this new table the data from our other table.

You may think ok I need to query the table, make an object then send that to the insert function that I create. But instead, we should group all these actions into one function. Perform multiple `execute()` operations as one transaction on the database to then `commit()`.

We can use a `SELECT` within the `INSERT` when putting in data. Ignoring the need for parameterized values since the data is already in the table. For this example, we will have an `archive_product` and `archive_review` table to move to.

1. Copy over product
2. Copy over reviews
3. Delete reviews
4. Delete product

```
def archive_product(product_id: int)-> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("""INSERT INTO archive_product
                        SELECT id, name, brand, price, stock_qty
                        FROM review WHERE id = ?""", (product_id,))

        cursor.execute("""INSERT INTO archive_review
                    SELECT id, product_id, rating, title, comment, reviewer
                    FROM review WHERE product_id = ?""", (product_id,))

        cursor.execute("DELETE FROM review WHERE product_id = ?",
                                                        (product_id,))

        cursor.execute("DELETE FROM product WHERE id = ?",
                                                        (product_id,))

        conn.commit()

        return True
    except Exception:
        conn.rollback()
        return False
    finally:
        conn.close()
```

With this situation we must move the reviews to a new table along with products. SQL only allows for a foreign key to reference one table. We can't have the reviews stay in their current table and move products to another table as we will get foreign key constraint errors.

This can be a high cost especially if an item has say, 1000 reviews. We will have to copy over the product, then copy over all those reviews and deleting all that data. It can be taxing on the database system. For systems that may not rely on foreign key constraints in this way moving rows of data may workout, but in this situation we may want to look at the next option, adding a new column to the table.

8.10.3 Soft Delete – Active/Inactive Column

In this situation where we have foreign keys the better design may to be add a column to our table to mark if something is active or not. You could have had this already designed in your table when you first created it.

```
CREATE TABLE IF NOT EXISTS product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    brand TEXT NOT NULL,
    price REAL NOT NULL,
    stock_qty INTEGER NOT NULL
    active INTEGER NOT NULL
);
```

But if, say you are in production and you already have all kinds of data. You can't really `DROP` your table (losing all the data) to recreate with a new column. For that we can use the `ALTER TABLE` command.

```
ALTER TABLE product ADD COLUMN is_active INTEGER DEFAULT 1;
```

Here we add a column and set that default values is '1'. All our existing rows of data will get filled with 1 for True when active. You may see we are not using `NOT NULL` here. This is because the `DEFAULT` command does just that for us, if a value is not provided it will automatically put in 1. Which most new products we `INSERT` are active items.

Once we update our table to have this column we can perform our "soft delete." This really isn't a delete but rather an UPDATE.

```
def deactivate_product(product_id : int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
           "UPDATE product SET is_active = 0 WHERE id = ?", (product_id,))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

I just created a function to directly set the `is_active` property to '0'. If we want to reactivate? Then we need another function to change it. Or as you may be thinking, let's make one function that SET the `is_active` to a value sent.

```
def update_is_active(product_id : int, active : int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("UPDATE product SET is_active = ? WHERE id = ?",
                                                  (active, product_id))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Either way works but typically if we are deactivating item it is an old product and we aren't selling anymore (unless we found some hidden stock in the warehouse).

What about reviews? They can stay where they are and we really don't need to add in a `is_active` column. Remember that we query the reviews based on the `product_id`. We just won't ever query those reviews unless looking at inactive products.

Update SELECT queries

We need to now revisit our `SELECT` statements. Not only do we have our standard WHERE clause in our queries but now we will need to always check that `is_active = 1`. This may be a single clause if we are selecting everything from a table or we will need to add in an `AND` to check multiple conditions.

```
cursor.execute("SELECT * FROM product WHERE is_active = 1")

cursor.execute("SELECT * FROM product WHERE id = ? AND is_active = ?",
                                                        (product_id,))
```

8.11 Advanced Database Operations

As applications grow larger, writing raw SQL for every database operation can become repetitive and error prone. For this reason, many professional Python applications use an Object-Relational Mapper (ORM).

All this code and functionality that we wrote usually is referred to "boilerplate code." That is code that repeats with little to no alteration. It provides the necessary structure to our application especially with database access. But this type of operation is the same across different applications. We are doing CRUD operations based on objects. Instead we can have an ORM that will map our Python objects to database tables for us!

8.11.1 What Is an ORM?

An Object-Relational Mapper (ORM) is a tool that allows you to interact with database tables using Python objects instead of writing SQL directly.

Conceptually:

- Database tables ↔ Python classes
- Table rows ↔ Python objects
- Columns ↔ Object attributes

Instead of writing SQL like:

```
SELECT * FROM product WHERE id = 1;
```

an ORM might allow you to write:

```
product = session.get(Product, 1)
```

Behind the scenes, the ORM translates your Python code into SQL and sends it to the database. Then translates the row of information automatically to the object.

8.11.2 Why ORMs Exist

ORMs solve several common problems:

- Reduce repetitive SQL code
- Centralize database logic
- Improve readability and maintainability
- Make it easier to change databases later
- Integrate validation and relationships directly into objects

They are especially common in web applications, where database access is frequent and complex.

The most widely used ORM in Python is the `SQLAlchemy` library. This is a 3rd party library that you would need to use the package installer(pip) to put within your project

For this course we are using the built-in sqlite3 library and the raw SQL commands. You still need to understand the use of SQL and what is happening "under the hood" of ORMs. If you choose to take the Python Web and API Development course we will explore the use of ORMs when building web applications.